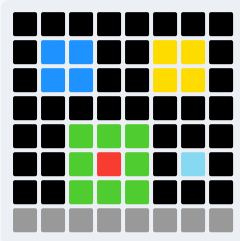


Can a small AI play ARC-AGI-3?



16 colours, 64×64

We wrapped a small, free AI model in a helper program and let it play puzzle games nobody explains. It played no better than pressing random buttons. Here is what we tried, what happened, and what we got wrong along the way.

A research write-up. Every test had its pass/fail rule written down *before* it ran, and every number here is re-checked against the saved results. [How we tested.](#)

THE GAME

Puzzles with no instructions

ARC-AGI-3 is a set of small video-game-like puzzles. The player sees a screen of coloured squares and a few buttons — arrow-like keys, and sometimes a click on a square (ACTION6 x y). Nobody says what the goal is. You press things, watch what changes, and work it out. People manage this easily. The best AI systems in the world still struggle.

Our question was narrow: **can a small AI model that we do not retrain**, helped by a smart program around it (a “harness”), do better than chance?

AT A GLANCE

Five experiments, one scoreboard

E0	Let the AI play Small model and one 8× bigger	■ NOT BETTER THAN RANDOM
E1	Give the AI a memory Skip buttons that did nothing; remember deaths	■ WEAK
E2	Show the AI its buttons first Test each button, then tell the AI what it does	■ MADE IT WORSE
E3	Make the AI label its moves “To learn” or “to win”, plus a diary of its experiments	■ MADE IT WORSE
E4	Smarter random, no AI Random buttons, minus the ones known to do nothing	■ WORKED ON PRACTICE GAMES
Build 3	Enter it in the competition Scored on 110 hidden games	■ NO CHANGE: 0.07

HOW WE TESTED

Practice games, test games, and the steps in between

Every experiment on this page went through the same process. That process matters as much as the results, because it decides which results can be trusted.

The games were split once, before any AI played

There are 25 public games. We shuffled them with a fixed number (20260912) so the split could not be nudged, and put 8 aside for **practice** and 17 aside for the **test**. The split has never changed.

8 practice games “tune” — try settings, find bugs, decide whether an idea is worth testing

dc22 ft09 ls20 s5i5 sc25 sk48 tn36 tu93

17 test games “held-out” — kept unseen until the verdict run; only these decide a result

ar25 bp35 cd82 cn04 g50t ka59 lf52 lp85 m0r0 r11l re86 sb26
sp80 su15 tr87 vc33 wa30

r11l (dashed) is reported but left out of the totals: its first level is so easy that random button-pressing can beat a human's time, so it would reward luck. That leaves **16 games** in every verdict.

110 hidden games the competition's own test — nobody outside the organisers can see them



Like a practice exam and a final exam

If you adjust an idea on the same games you grade it on, you end up fitting those games by luck. Its score looks better than it really is.

It caught us once

Adding coordinates to the screen looked like it fixed clicking on 2 practice games. On the test games it did nothing.

Even the test games aren't enough

Smarter random (E4) won on the 17 test games, then scored no better on the 110 hidden ones. Only the hidden games test whether an idea carries over.

The steps an experiment climbs

Each step is cheaper than the next, and each is allowed to say more. An experiment stops as soon as a step says stop. Steps 1–4 only ever use practice games.

1

Smoke

a few games

Does the test program run from start to finish?

Install, load the AI, play, save the results.
Checks the setup only.

■ PROVES NOTHING

2

Diagnose

2 practice games

Why is something going wrong?

A before-and-after on one suspected cause, such as the AI's clicking.

■ POINTS TO A FIX

3	Probe 2-3 practice games, 1 try	Does the new helper actually switch on? Replies can be read, the AI picks real buttons, nothing is broken.	A MECHANICS CHECK
4	Tune 8 practice games, 3 tries	Is this idea worth a real test? A go/stop rule, written before the run. This is where E2 and E3 stopped.	GO OR STOP ONLY
5	Test "held-out" · 16 test games	Does the idea beat the comparison? The only step whose numbers count as a result.	THE VERDICT
6	Competition 110 hidden games	Does it carry over to games nobody has seen? "Abandoned at tune" (E2) and "stopped at tune" (E3) mean the same thing: the rule said stop, so neither idea was ever run on the test games. Both stopped for the same reason — the AI wasted more moves with the new helper, not fewer.	DOES IT CARRY OVER

Which steps each experiment went through

EXPERIMENT	SMOKE	DIAGNOSE	PROBE	TUNE	TEST	COMPETITION
E0 · let the AI play	ran	ran	ran	—	ran	—
E1 · memory	—	—	ran	—	ran	—
E2 · explain the buttons	—	—	—	stopped	—	—
E3 · learn or win	—	—	—	stopped	—	—
E4 · smarter random	—	—	—	—	ran	Build 3

Steps were skipped when an earlier run had already covered them: E1 was built on E0's working setup, so it started at the probe. E4 uses no AI and runs on a laptop in minutes, so it went straight to the test games. Plain random was entered first as **Build 1** (0.07), the score every later entry had to beat.

Tries (seeds): how many times each game was played

Random choices and the AI's own replies both involve chance, so one run can be lucky. Each game is played several times, and each try is numbered — try 0, try 1, try 2 and so

on. The number sets the dice, so **try 3 always plays the same way**. That lets two players be compared try by try on identical luck, and lets one big run be split across two machines without changing a single move.

1-2

Diagnose and probe:
enough to see
whether it works at all

3

Tune on practice
games: enough for a
go/stop call

5

Test verdicts with the
AI: the project's
minimum for a result

20

Smarter random (E4):
wins are rare and a
laptop is cheap

Two honest caveats. The AI server doesn't always give exactly the same reply to the same prompt, so AI runs repeat almost exactly, not perfectly. And the project's rule says at least 5 tries; tune used 3 because it can only stop an idea, never prove one. The written rule doesn't say that exception out loud yet.

▼ Every run on Kaggle, and which step it was

KAGGLE RUN (ARC-AGI-3-...)	EXPERIMENT	STEP	GAMES × TRIES × MOVES
build-1-random	—	Competition	110 hidden · plain random, 80 moves → 0.07
e0-smoke	E0	Smoke	pipeline check
e0-diag	E0	Diagnose	dc22, ls20 × 1 × 80 · the “before”
e0-diag2	E0	Diagnose	dc22, ls20 × 2 × 80 · with coordinates
e0-gemma4-e4b	E0	Test	17 × 5 × 80 · small model
e0-qwen3-8b	E0	Test	17 × 5 × 80 · setup failure, not a result
e0-probe-gemma4-31b	E0	Probe	ls20, tu93 × 1 × 20 · big model
e0-gemma4-31b-e012	E0	Test	17 × tries 0-2 × 80
e0-gemma4-31b-e34	E0	Test	17 × tries 3-4 × 80 · one run split in two to fit Kaggle's 9-hour limit
e1-probe-e4b	E1	Probe	dc22, sc25, tu93 × 1 × 80
e1-heldout-e4b	E1	Test	17 × 5 × 80 · compared against E0's run
e2-tune-e4b	E2	Tune → stop	8 × 3 × 80 · six conditions
e3-tune-e4b	E3	Tune → stop	8 × 3 × 80 · version 1 crashed and played nothing

KAGGLE RUN (ARC-AGI-3-...)	EXPERIMENT	STEP	GAMES × TRIES × MOVES
build-3-random-prune	E4	Competition	110 hidden · smarter random, 300 moves → 0.07

E4's test run isn't listed: it ran on a laptop, 17 games × 20 tries at 80 and 300 moves.

E0 · THE BASELINE

■ NOT BETTER THAN RANDOM

The AI played no better than random buttons

We let the AI choose every move from a text picture of the screen. **It cleared no levels.** Pressing random buttons cleared one. A model eight times bigger also cleared none.

- The AI was fine with arrow keys. It went wrong on **clicks**: it kept clicking squares that do nothing.
- The bigger model wasted fewer clicks, but still did not beat random.
- A third model (Qwen3-8B) broke the rules of the game setup — it kept choosing buttons the game did not have — so its result says nothing about how well it plays.

▼ Technical detail

17 held-out games (16 count toward the verdict), 80 actions per episode, 5 seeds. “Screens per 100” is how many different screens the player saw per 100 moves — a measure of exploration.

16 VERDICT GAMES	RANDOM	GEMMA-4-E4B	GEMMA-4-31B
Levels per game	0.0125	0.000	0.000
Competition score (RHAE)	0.0348	0.0000	0.0000
Screens per 100 moves	78.5	68.6	73.2
Same, click games only	73.5	59.6	66.1

The 31B run had 7 parse failures in 6,800 model calls, so its result reflects the model, not a broken prompt. Qwen3-8B had 7 of 85 episodes excluded under the pre-registered 20% parse-failure rule. Cost: about 13.6 GPU-hours on Kaggle.

E1 · MEMORY

■ WEAK

A memory helped the AI explore, not win

We added two helpers. One **blocks buttons that just did nothing** on that screen. The other **writes a note after each death** so the AI can avoid it next time.

- The blocking helper made the AI explore more of each game. The death notes did nothing.
- It cleared one level — tying random, not beating it. That counts as **weak** under the rule we set beforehand.
- That one clear mostly came from the helper swapping in random moves, not from the AI's own choices.

▼ Technical detail

16 VERDICT GAMES	RANDOM	NO HELPERS	BLOCKING	NOTES	BOTH
Level clears	1	0	1	0	1
Screens per 100 moves	78.5	68.6	74.4	69.6	74.5

12.1% of all moves were substitutions; in the clearing episode, 64% were. The notes helper wrote 34 rules, but the control repeated a death only once in 85 episodes, so it had almost nothing to fix. Cost: about 2.9 GPU-hours.

E2 · BUTTON CHECK

■ MADE IT WORSE

Explaining the buttons made the AI waste more moves

Before the AI played, the helper pressed a few buttons itself and wrote down what each one changed. **With that list, the AI wasted more of its moves** on buttons that do nothing — the opposite of what we hoped.

- On one game it roughly doubled the wasted moves. On a few others it helped the AI explore — but never in the way the idea needed.
- We stopped at the practice stage, as our rule said to.
- We cannot tell whether the idea is wrong or our version of it was too crude. We said so before running it.

▼ Technical detail

Dead-move rate = the share of the AI's own moves that changed nothing. The comparison player got the same number of extra moves, with no button list. Lower is better.

PRACTICE GAMES, 3 SEEDS	5 TEST MOVES	10	20
With the button list	0.344	0.321	0.333
Without (extra moves only)	0.284	0.303	0.293

Cost: about 2.7 GPU-hours.

E3 • LEARN OR WIN

■ MADE IT WORSE

Labelling moves “to learn” or “to win” didn’t help either

We asked the AI to start every move with **LEARN** (to find out how the game works) or **WIN** (to make progress). After each LEARN move, the helper wrote down what actually happened, and the AI saw its last five “experiments”. This time **the AI chose what to test**, not the helper.

- With the labels, the AI **wasted more moves** — the same direction as E2. We stopped at the practice stage, as our rule said.
- It mostly **ignored the instruction**: two replies in three had no label at all.
- So this is a weak test: it shows this AI doesn’t reliably follow the format, not that the idea is bad.

▼ Technical detail

PRACTICE GAMES, 3 SEEDS	WITH LABELS	WITHOUT
Dead-move rate (lower is better)	0.342	0.295
Screens per 100 moves	59.0	61.7
Level clears	0	0

Of 1,920 replies: 385 LEARN, 255 WIN, 1,280 unlabelled. The same game as in E2 (sk48) carried the loss. This idea had been set aside at the start because other teams have already published it on this benchmark; we ran it anyway to close the question. Cost: about 1 GPU-hour.

E4 • SMARTER RANDOM • BUILD 3

■ WORKED ON PRACTICE GAMES

Smarter random doubled the clears — then vanished on hidden games

We dropped the AI entirely. A player presses random buttons, but **never repeats a button it has already seen do nothing** on that screen. On the practice games it cleared **twice as many levels** as plain random.

Levels cleared on 16 practice games, 20 tries each, 300 moves

Plain random		19
Smarter random		38

Kaggle score on 110 hidden games

Build 1 (random)		0.07
Build 3 (smarter)		0.07

Each group has its own scale. Top: bar length is levels cleared (0–38). Bottom: bar length is score (0–0.07).

- It won on **18 of 20 tries** and lost none — very unlikely to be luck.
- But almost all of the gain came from **one game** (lp85): cleared every time, against once for plain random.
- We put it into our competition entry. On Kaggle’s hidden games it scored **0.07** — **exactly the same** as the old plain-random entry.

▼ Technical detail

17 HELD-OUT GAMES, 20 SEEDS	RANDOM	SMARTER	WON/LOST/TIED
Level clears, 300 moves	19	38	18/0/2
Level clears, 80 moves	6	12	6/0/14
RHAE, all 17 games, 300 moves	0.0916	0.1000	

One-sided sign test at 300 moves: $p < 0.0001$. lp85 level 1 cleared in 20/20 tries against 1/20. Kaggle reports scores to two decimals, so a change smaller than 0.01 would not show. Runs on a laptop CPU; the competition entry uses one daily submission.

What went wrong, and how we caught it

A bug made random play look better than it was

Our test program reused the same game for every try. Resetting it kept the level you had reached, so after one real win, every later try started ahead and got counted as another win. We spotted it because some levels seemed to be cleared in a single move, which is impossible. **Fixing it lowered random's score from 0.037 to 0.0125 levels per game.** The AI results were not affected, and no conclusion flipped — but we corrected every place the wrong number appeared.

A result from two games did not hold up on thirteen

Adding coordinates to the screen picture looked like it fixed the clicking problem on two practice games. Across thirteen real test games it did nothing. We now never claim an effect from practice games alone.

A measurement meant something different than it looked

One number seemed to show the AI stuck pressing one button. It was really counting the button's *name*, so clicking a different square every time looked identical. Reading it next to exploration fixed the misreading.

A test package was missing a file

One run on Kaggle crashed before playing, because a file the program needed wasn't packaged with it. Nothing was measured from it. The build check now catches any missing file before a run is sent.

The same question did not always give the same answer

Even with fixed random seeds, the model server sometimes answered identical prompts slightly differently. Our pass/fail rules already required beating random too, so this could not create a false win.

READING IT CAREFULLY

What this shows — and what it doesn't

It shows

- This AI, this setup, 80 moves: no better than random.
- Its main failure is clicking squares that do nothing.

It doesn't show

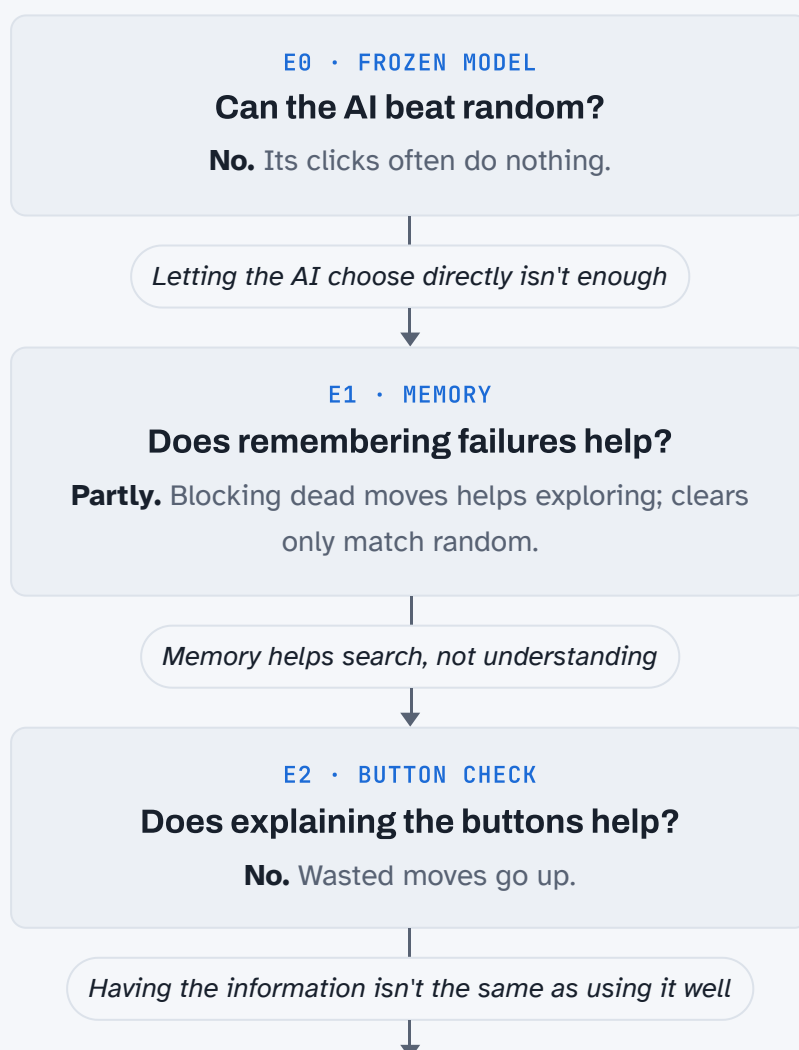
- That small AI models *can't* do this — only this setup.
- That “find what you control first” is a wrong idea.

- Blocking do-nothing moves helps exploration, for AI and random alike.
- Explaining the buttons up front does not help this AI — nor does a diary of its own experiments.
- Extra instructions can make this AI worse, and it follows a new reply format only a third of the time.
- That “learn vs win” moves are a bad idea — the AI barely used the labels.
- That remembering deaths is useless — deaths barely happened.
- That smarter random helps on unseen games. It didn't.

WHERE THIS LEADS

From five experiments to the next two

Each experiment left one lesson, and each lesson set up the next question. The last one points to a gap that smarter random can't fill. Before trying to fill it, one cheap step makes sure the measurements can be trusted. Then there are two ways to try.



E3 • LEARN OR WIN

Can the AI separate learning moves from winning moves?

Stopped at practice. Wasted moves go up; labels mostly ignored.

Labels and a diary don't give the AI a way to experiment

E4 • SMARTER RANDOM

How far can a simple rule go?

Doubles clears on test games; the hidden score doesn't move.

Simple search works on known games but doesn't carry over

THE GAP AFTER E4

Skipping dead moves can't work out how a game works

It can't infer the rules, the goal, cause and effect, or a plan.

First, make sure the measurements can be trusted

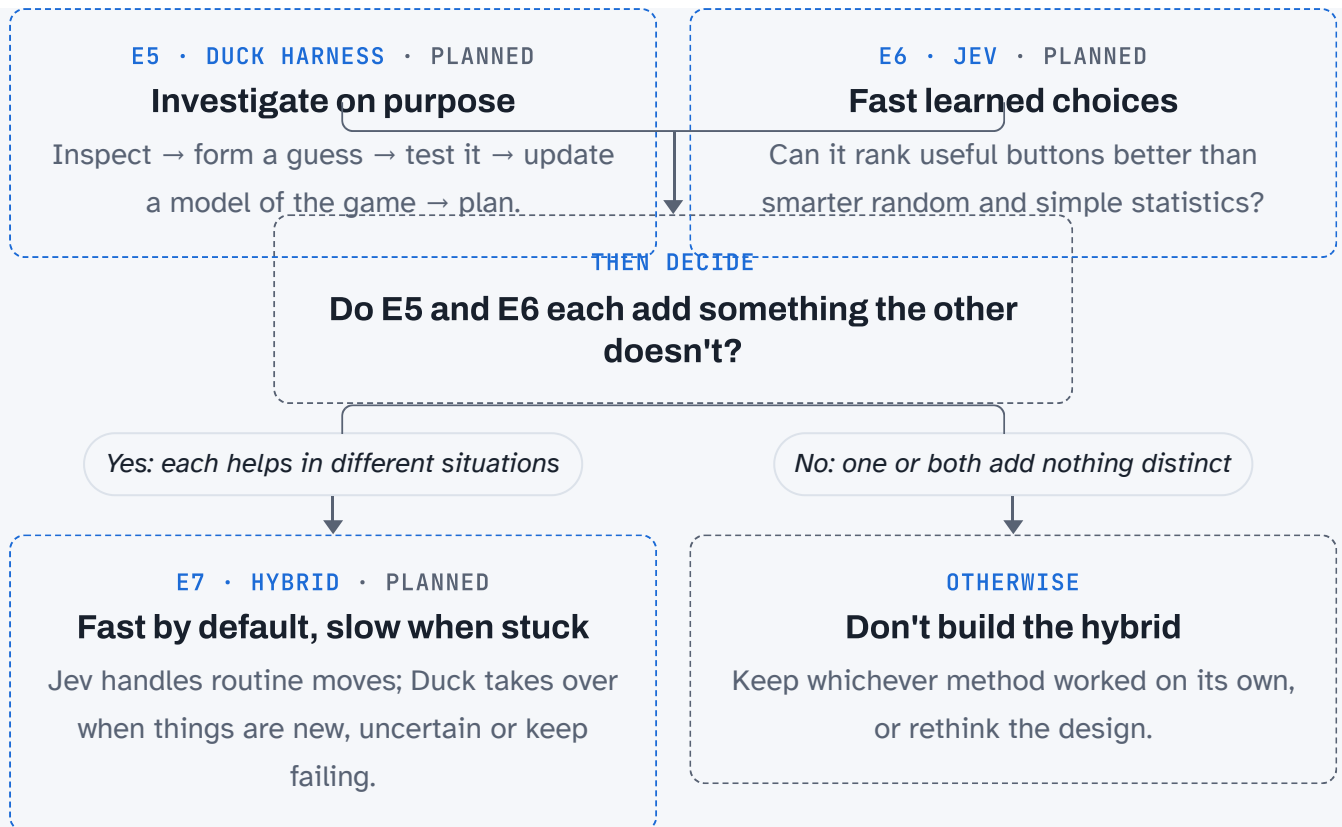
P1 • GROUNDWORK, FIRST • PLANNED

Ignore on-screen clocks, and add the safety checks

Many games show a clock that changes every move, so a useless move looks useful. Fix that "did anything change?" check, then see whether smarter random clears more. Laptop only, under an hour. Also builds the checks against another replay bug.

Can slow, careful reasoning fill the gap?

Or is a better fast choice of button enough?



Dashed boxes are plans. None of P1, E5 or E6 has a written pass/fail rule yet, and none will run until it does. P1 comes first because every later number depends on the “did anything change?” check. E5 and E6 must beat the best simple method available when they run — smarter random at least.

STILL OPEN

Ideas worth testing next

- **Many more moves per game.** The competition allows far more than 300; only then do repeated deaths become common enough to learn from.
- **Ignore on-screen clocks.** Many games show a timer that changes every move, which hides whether a move really did anything.
- **Think in objects, not squares.** Block “clicking the wall”, not just one square of it.
- **A different way for the AI to see and act.** The team that won the first ARC-AGI-3 checkpoint used a larger free model that writes small programs against the game — with objects already picked out and before/after views — and scored far higher. Comparing with that is our next step.

- **A different kind of controller** that aims to keep something on screen at a target, instead of picking buttons one by one.

Cost in total: about 20 GPU-hours on Kaggle, plus laptop CPU for the random-play experiments.

The full technical record — every rule, run, correction and file — lives in the project's experiment register and write-up. Results cover the 25 public ARC-AGI-3 games (8 for practice, 17 held back for testing) and one competition submission.